Интерпретирање међујезика

- Програмско генерисање међујезика
- Анализа међујезика
- Интерпретирање међујезика

Анализа и интерпретирање међујезика

- Пример: анализа и интерпретирање праволинијских програма исказаних у SLP језику
  - SLP (Straight-Line Programming language)
  - Добар увод у
    - Окружења (табеле симбола, које пресликавају имена променљивих у информацију о њима)
    - Апстрактну синтаксу (структура података, која представља структуру фраза у програмима)
    - Рекурзију преко структура података у облику стабала
    - Функционални стил програмирања без исказа доделе
  - Добро загревање за програмирање у циљном језику

Улаз

- Програми који се интерпретирају претходно се трансформишу у стабло апстрактне синтаксе
  - Ту трансформацију (parsing) ћемо радити касније
    - Сада пишемо циљни програм директно помоћу конструктора

```
IrStm* progT() {
  // a := 5 + 3; b := a - 1; print(b);
  return
    makeCompoundStm(
      makeAssignStm(
        "a", makeOpExp(makeNumExp(5), OK_PLUS, makeNumExp(3))),
      makeCompoundStm(
        makeAssignStm(
          "b", makeOpExp(makeIdExp("a"), OK_MINUS, makeNumExp(1))),
        makePrintStm(makeLastExpList(makeIdExp("b")))
      )
    );
}
```

Улаз

```
IrStm* prog1() {
  // a := 5+3; b := (print(a, a-1), 10*a); print(b);
  return
    makeCompoundStm(
      makeAssignStm("a", makeOpExp(makeNumExp(5), OK_PLUS, makeNumExp(3))),
      makeCompoundStm(
        makeAssignStm("b", makeEseqExp(
          makePrintStm(makePairExpList(makeIdExp("a"), makeLastExpList(
            makeOpExp(makeIdExp("a"), OK_MINUS, makeNumExp(1))))),
          makeOpExp(makeNumExp(10), OK_MUL, makeIdExp("a")))),
        makePrintStm(makeLastExpList(makeIdExp("b")))
      )
    );
}
```

Улаз

```
IrStm* prog1() {
  // a := 5+3; b := (print(a, a-1), 10*a); print(b);
  return
    makeCompoundStm(
      makeAssignStm("a", makeOpExp(makeNumExp(5), OK_PLUS, makeNumExp(3))),
      makeCompoundStm(
        makeAssignStm("b", makeEseqExp(
          makePrintStm(makePairExpList(makeIdExp("a"), makeLastExpList(
            makeOpExp(makeIdExp("a"), OK_MINUS, makeNumExp(1))))),
          makeOpExp(makeNumExp(10), OK_MUL, makeIdExp("a")))),
        makePrintStm(makeLastExpList(makeIdExp("b")))
      )
    );
}
```

Уланчава исказе

```
makeCompoundStm(s1, makeCompoundStm(s2, makeCompoundStm(s3, s4)));
```

Улаз

```
IrStm* prog1() {
  // a := 5+3; b := (print(a, a-1), 10*a); print(b);
  return
    makeCompoundStm(
      makeAssignStm("a", makeOpExp(makeNumExp(5), OK_PLUS, makeNumExp(3))),
      makeCompoundStm(
        makeAssignStm("b", makeEseqExp(
          makePrintStm(makePairExpList(makeIdExp("a"), makeLastExpList(
            makeOpExp(makeIdExp("a"), OK_MINUS, makeNumExp(1))))),
          makeOpExp(makeNumExp(10), OK_MUL, makeIdExp("a")))),
        makePrintStm(makeLastExpList(makeIdExp("b")))
      )
    );
}
```

Улаз

```
IrStm* prog1() {
  // a := 5+3; b := (print(a, a-1), 10*a); print(b);
  return
    makeCompoundStm(
      makeAssignStm("a", makeOpExp(makeNumExp(5), OK_PLUS, makeNumExp(3))),
      makeCompoundStm(
        makeAssignStm("b", makeEseqExp(
          makePrintStm(makePairExpList(makeIdExp("a"), makeLastExpList(
            makeOpExp(makeIdExp("a"), OK_MINUS, makeNumExp(1))))),
          makeOpExp(makeNumExp(10), OK_MUL, makeIdExp("a")))),
        makePrintStm(makeLastExpList(makeIdExp("b")))
      )
    );
}
```

Улаз

```
IrStm* prog1() {
  // a := 5+3; b := (print(a, a-1), 10*a); print(b);
  return
    makeCompoundStm(
      makeAssignStm("a", makeOpExp(makeNumExp(5), OK_PLUS, makeNumExp(3))),
      makeCompoundStm(
        makeAssignStm("b", makeEseqExp(
          makePrintStm(makePairExpList(makeIdExp("a"), makeLastExpList(
            makeOpExp(makeIdExp("a"), OK_MINUS, makeNumExp(1))))),
          makeOpExp(makeNumExp(10), OK_MUL, makeIdExp("a")))),
        makePrintStm(makeLastExpList(makeIdExp("b")))
      )
    );
}
```

Интерпретери без ивичних ефеката

- ...су добар увод у семантику и граматику језика
- „Без ивичних ефеката” значи без исказа доделе
  - за ажурирање променљивих и структура података
- Вредност се додељује само при иницијализацији
  - За локалне промењиве: иницијализационе декларације
    - Нпр. int i=j+3;

Пример анализе међујезика
Задатак

- Написати функцију **int maxargs(IrStm*)**, која враћа највећи број аргумената било ког print исказа унутар било ког подизраза у датом исказу
  - Нпр. maxargs(prog1()) је 2
  - Упутство: print исказ може да садржи исказе, који могу да садрже друге print исказе
    - То је посредна рекурзија
- Напомена: Школске верзије кодова које су дате на слајдовима, у књизи и у приложеним материјалима не рукују грешкама и не чисте динамички заузету меморију.

Пример анализе међујезика

Решење (1/3): Анализа исказа

```
int stmargs(IrStm* x) {
  switch (x->kind) {
    case SK_COMPOUND: {
      int n = stmargs(static_cast<CompoundStm*>(x)->stm1);
      int m = stmargs(static_cast<CompoundStm*>(x)->stm2);
      return max(n, m);
    }
    case SK_ASSIGN:
      return expargs(static_cast<AssignStm*>(x)->exp);
      break;
    case SK_PRINT:
      return explistargs(static_cast<PrintStm*>(x)->exps);
    }
  return -1; // error
}
```

Пример анализе међујезика

Решење (2/3): Анализа израза

```
int expargs(IrExp* x) {
  switch (x->getKind()) {
    case EK_ID:
    case EK_NUM:
      return 1;
    case EK_OP: {
      int n = expargs(static_cast<OpExp*>(x)->left);
      int m = expargs(static_cast<OpExp*>(x)->right);
      return max(n, m);
    }
    case EK_ESEQ: {
      int n = stmargs(static_cast<EseqExp*>(x)->stm);
      int m = expargs(static_cast<EseqExp*>(x)->exp);
      return max(n, m);
    }
  }
  return -1; // error
}
```

Пример анализе међујезика

Решење (3/3): Анализа листе израза и maxargs

```
int explistargs(IrExpList* x) {
  switch (x->getKind()) {
    case ELK_PAIR: {
      PairExpList* = static_cast<PairExpList*>(x);
      int n = expargs(p->head);
      int m = explistargs(p->tail) + 1;
      return max(n, m);
    }
    case ELK_LAST:
      return expargs(static_cast<LastExpList*>(x)->exp);
  }
  return -1; // error
}


int maxargs(IrStm* x) {
  return stmargs(x);
}
```

SLP интерпретер
Задатак са упутством (1/4)

- Написати функцију **void interp(IrStm*)**, која интерпретира задати SLP програм. Упутство:
  - Направити две међусобно рекурзивне функције interpStm и interpExp
  - Представити табелу пресликавања идентификатора у њима придружене вредности типа int, као листу парова (id, value)

```
struct Table {string id; int value; Table* tail;};

Table* makeTable(string id, int value, Table* tail) {
  return new Table(id, value, tail);
}
```

SLP интерпретер
Задатак са упутством (2/4)

- Празна табела се представља са nullptr
- Прототип функције interpStm:

```
Table* interpStm(IrStm* s, Table* t);
```

- interpStm узима табелу *t1* и производи *t2* у којој се неки идентификатори пресликавају у друге вредности
  - То је резултат извршавања исказа
  - Нпр. нека *t1* пресликава a у 3 и c у 4: {a→3, c→4}
    - Листа изгледа овако: vrh->[a,3]->[c,4]
  - Нека је *t2* као *t1*, али пресликава c у 7: {a→3, c→7}
  - То се може овако записати: t2 = update(t1, c, 7)
  - Ако претпоставимо да се увек узима прва појава c, *t2* се може добити од *t1* тако што се на почетак *t1* дода нови пар (c,7)
    - Нова листа: vrh->[c,7]->[a,3]->[c,4]

SLP интерпретер
Задатак са упутством (3/4)

- Функција update само додаје нов пар на врх листе
- Функција lookup тражи прву појаву key од почетка листе
  - Прототип: `int lookup(Table* t, string key);`
- Интерпретирање израза је сложеније јер враћа вредност и има ивичне ефекте
  - Циљ: симулирати SLP исказ доделе без ивичних ефеката унутар интерпретера (наравно print ће имати ивичне ефекте - приказе)
  - Решење: резултат interpExp је нова табела IntAndTable

```
struct IntAndTable {int res; Table* t;};

IntAndTable interpExp(IrExp* e, Table* t);
IntAndTable interpExpList(IrExpList* el, Table* t);
```

SLP интерпретер
Задатак са упутством (4/4)

- Резултат интерпретирања израза *e1* са табелом *t1* је целобројна вредност *i* са новом табелом *t2*
  - Приликом интерпретирања израза са два подизраза (нпр. OpExp), табела *t2*, која резултује из обраде првог подизраза, се користи за обраду другог подизраза

SLP интерпретер
Решење (1/4): интерпретирање исказа

```
Table* interpStm(IrStm* x, Table* t) {
  switch (x->kind) {
    case CompoundStm: {
      CompoundStm* p = static_cast<CompoundStm*>(x);
      Table* to = interpStm(p->stm1, t);
      return interpStm(p->stm2, to);
    }
    case AssignStm: {
      AssignStm* = static_cast<AssignStm*>(x);
      IntAndTable wit = interpExp(p->exp, t);
      return makeTable(p->id, wit->i, wit->t); // add var.
    }
    case PrintStm:
      PrintStm* p = static_cast<PrintStm*>(x);
      IntAndTable wit = interpExpList(p->exps, t);
      return wit->t;
  }
  return nullptr; // error
}
```

SLP интерпретер

Решење (2/4): интерпретирање израза (1/2)

```
IntAndTable interpExp(IrExp* x, Table* t) {
  switch(x->kind) {
    case IdExp: {
      int i = lookup(t, static_cast<IdExp*>(x)->id);
      return IntAndTable(i, t);
    }
    case NumExp: {
      int i = static_cast<NumExp*>(x)->num;
      return IntAndTable(i, t);
    }
    case OpExp: {
      OpExp* p = static_cast<OpExp*>(x);
      IntAndTable wit = interpExp(p->left, t);
      int l = wit->i;
      wit = interpExp(p->right, wit->t);
      int r = wit->i;
      ...
```

SLP интерпретер

Решење (3/4): интерпретирање израза (2/2)

```
        ...
        switch (p->operKind) {
          case OK_PLUS:
            return IntAndTable(l + r, wit->t);
          case OK_MINUS:
            return IntAndTable(l - r, wit->t);
          case OK_MUL:
            return IntAndTable(l * r, wit->t);
          case OK_DIV:
            return IntAndTable(l / r, wit->t);
        }
      }
      case EseqExp:
        EseqExp* p = static_cast<EseqExp*>(x);
        Table* wt = interpStm(p->stm, t);
        return interpExp(p->exp, wt);
      }
    return IntAndTable(); // error
}
```

SLP интерпретер
Решење (4/4): интерпретирање листе исказа

```
IntAndTable interpExpList(IrExpList* x, Table* t) {
  switch (x->kind) {
    case PairExpList: {
      PairExpList* p = static_cast<PairExpList*>(x);
      IntAndTable ito = interpExp(p->head, t);
      std::cout << ito->i;
      return interpExpList(p->tail, ito->t);
    }
    case LastExpList: {
      LastExpList* p = static_cast<LastExpList*>(x);
      IntAndTable ito = interpExp(p->last, t);
      std::cout << ito->i << std::endl;
      return ito;
    }
  }

  return IntAndTable(); // error
}
```

SLP интерпретер
Домаћи

- Завршити SLP интерпретер
  - Треба направити функцију lookup за табеле